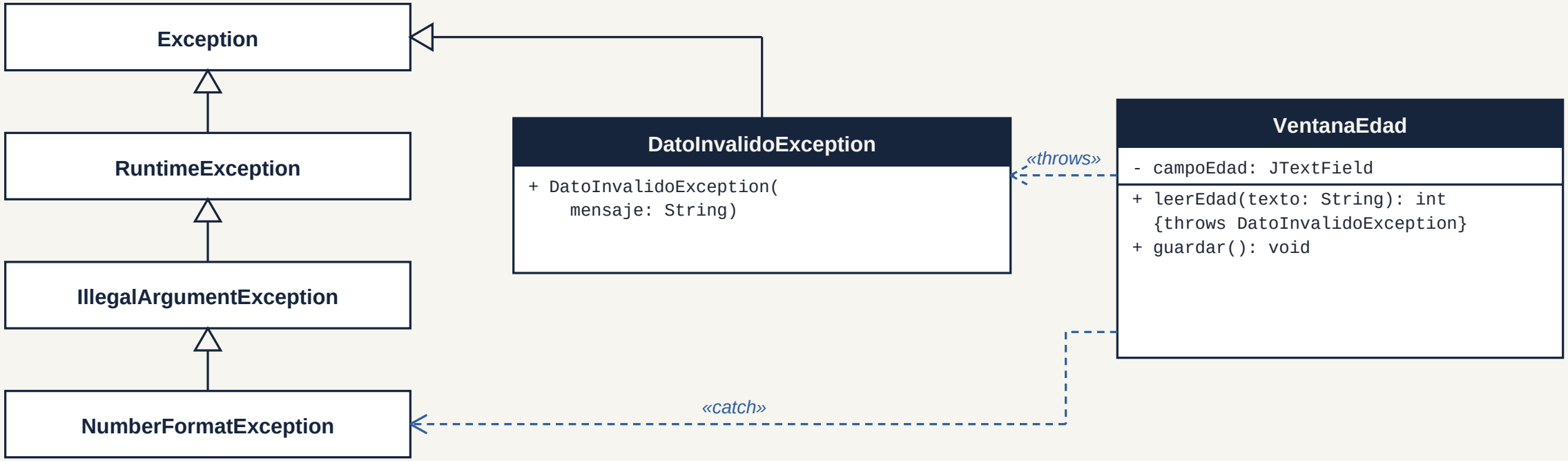
PRÁCTICA 10 · PROGRAMACIÓN ORIENTADA A OBJETOS

Excepciones más comunes en GUI

Ilustración del relanzamiento de excepciones

Equipo: [5] · Grupo [_7_]

Diagrama UML de clases

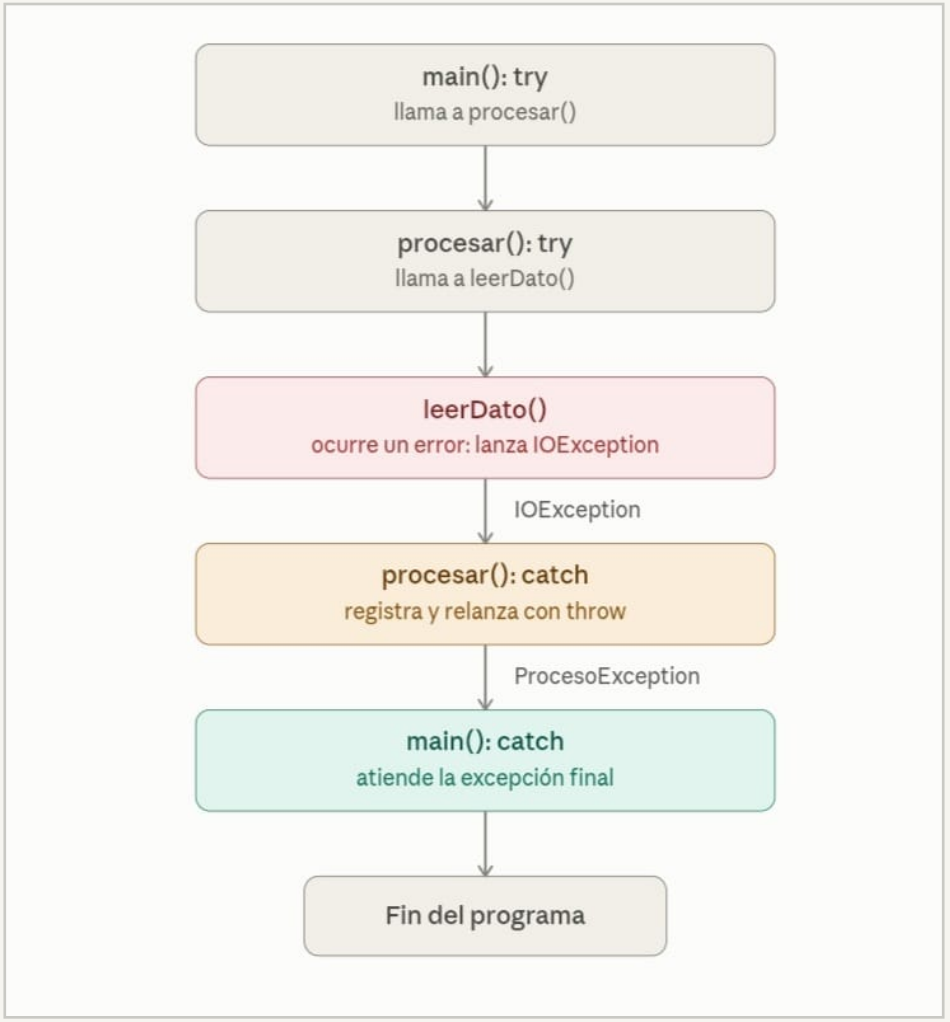


Marcada: Exception y DatoInvalidoException. No marcada: RuntimeException y sus subclases (NumberFormatException).

Herencia (extends)

Dependencia (usa, captura o lanza)

Flujo del relanzamiento de la excepción



Pseudocódigo del relanzamiento

MÉTODO leerEdad

```
ALGORITMO leerEdad(texto) -> entero
INICIO
  INTENTAR
    edad <- convertirAEntero(texto)
    RETORNAR edad
  CAPTURAR errorFormato
    ESCRIBIR "Error: dato inválido"
    // Forma 1: relanzar la misma
    LANZAR errorFormato
    // Forma 2: lanzar una propia
    LANZAR NUEVO DatoInvalido(
        "La edad debe ser un número")
FIN
```

BOTÓN Guardar

```
ALGORITMO botonGuardar
INICIO
  INTENTAR
    edad <- leerEdad(campoEdad.texto)
    ESCRIBIR "Edad guardada: ", edad
  CAPTURAR DatoInvalido
    MOSTRAR DIÁLOGO(error.mensaje)
FIN
```

FLUJO CON LA ENTRADA “abc”

convertirAEntero falla → leerEdad captura y relanza → el botón atrapa y muestra el mensaje al usuario.

¿Qué es relanzar una excepción?

Capturar la excepción con catch, hacer algo con ella y volver a lanzarla para que otro método la atienda.

PASO 1

`try`

Ocurre el error dentro del bloque try.

→

PASO 2

`catch`

Se captura la excepción y se hace algo local: registrar o limpiar.

→

PASO 3

`throw`

Se lanza de nuevo para que el método que llamó decida qué hacer.

Forma 1: relanzar la misma excepción

```
public int leerEdad(String texto) {
    try {
        return Integer.parseInt(texto);
    } catch (NumberFormatException e) {
        System.out.println("Error: dato inválido");
        throw e;
    }
}
```

1. parseInt lanza NumberFormatException si el texto no es un número.
2. catch la captura y registra el error.
3. throw e; envía el mismo objeto al método que llamó.
4. Al ser no marcada, no hace falta throws.

Forma 2: relanzar una excepción propia

```
// 1) Excepción propia: hereda de Exception
public class DatoInvalidoException extends Exception {
    public DatoInvalidoException(String mensaje) {
        super(mensaje);
    }
}
```

```
// 2) El método declara throws y lanza la nueva excepción
public int leerEdad(String texto) throws DatoInvalidoException {
    try {
        return Integer.parseInt(texto);
    } catch (NumberFormatException e) {
        throw new DatoInvalidoException("La edad debe ser un número");
    }
}
```

Ejemplo en una ventana: botón Guardar

El usuario escribe “abc” en el campo de edad:

1. getText() devuelve “abc”.
2. parseInt lanza NumberFormatException.
3. El catch de leerEdad la captura y lanza DatoInvalidoException.
4. La excepción sube hasta el botón.
5. El catch del botón muestra el mensaje.

```
// Dentro del botón "Guardar"
try {
    int edad = leerEdad(campoEdad.getText());
    System.out.println("Edad guardada: " + edad);
} catch (DatoInvalidoException e) {
    JOptionPane.showMessageDialog(null, e.getMessage());
}
```

Resumen

Cuándo relanzar

Cuando el método puede reaccionar (registrar, limpiar) pero no resolver el error.

Cómo relanzar

throw e; reenvía la misma excepción. throw new ... lanza una excepción propia.

En una GUI

La capa de la ventana atrapa la excepción final y avisa al usuario.

Fuente: MADO-22, Guía práctica de estudio 10: Excepciones y errores